INFORME DE PRÁCTICA DE LABORATORIO - SESIÓN 7

Seguridad Distribuida y Control de Acceso (OAuth2 Resource Server y JWT)

1. DATOS DEL ESTUDIANTE

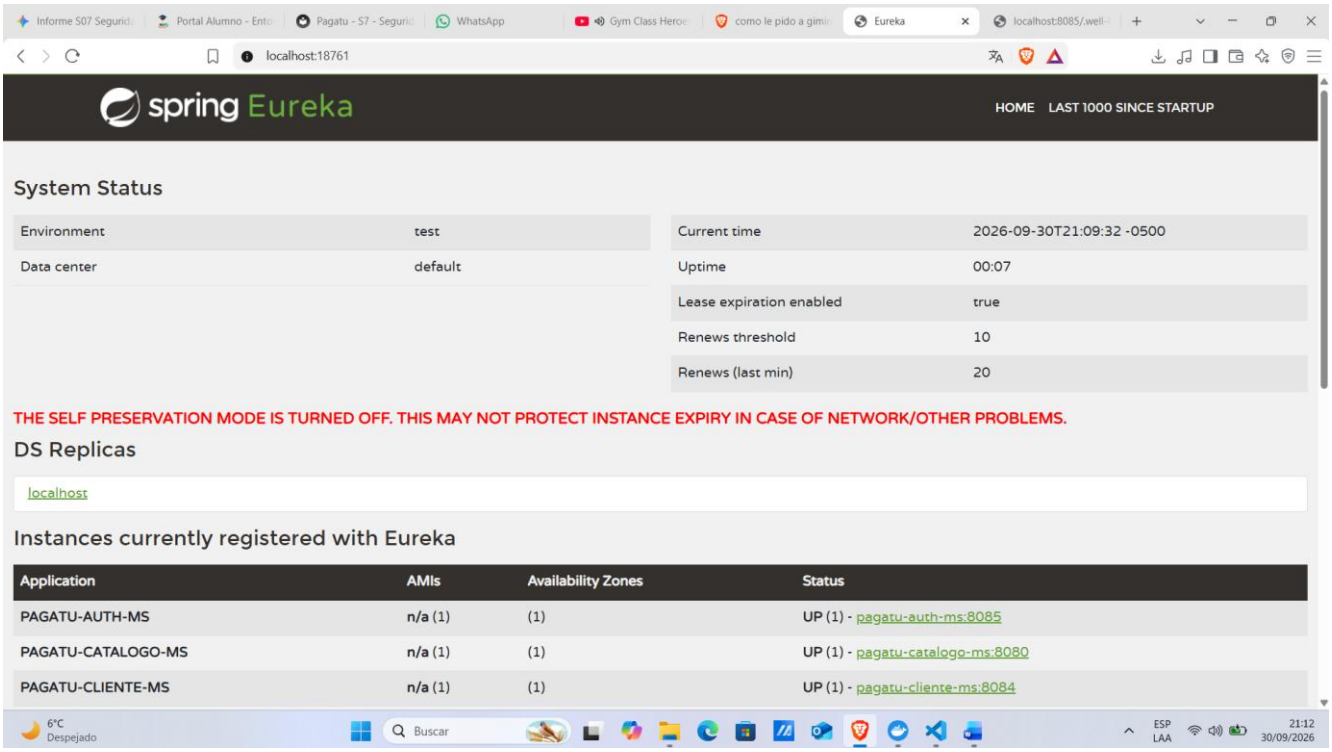
Nombre y Apellidos:	Witmer Jhetly Mamani Cahuana
Equipo de Trabajo:	Equipo 01
Sesión:	S07 Seguridad distribuida y control de acceso
Rol o Aporte Realizado:	Desarrollé el microservicio pagatu-auth-ms conectándolo a PostgreSQL y gestionando los esquemas con Flyway. Implementé la generación y firma de tokens JWT asimétricos con RS256, exponiendo el endpoint JWKS (/.well-known/jwks.json). También configuré pagatu-gateway y pagatu-orden-ms para operar como Resource Servers, retiré el idCliente del payload de las órdenes para extraerlo directamente desde el token, y apliqué la configuración de seguridad autónoma con RBAC en pagatu-cliente-ms.
Enlace del Repositorio en GitHub:	https://github.com/Witmer17/JWT.git

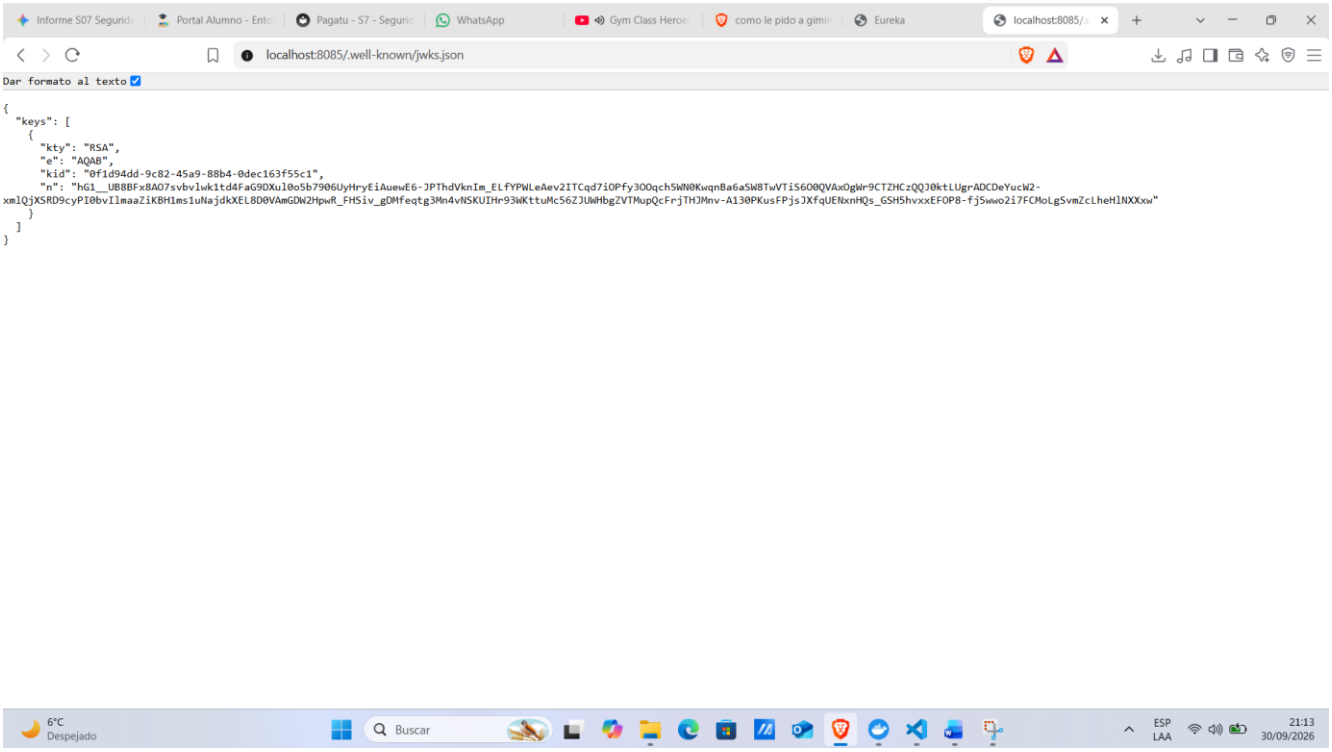
2. EVIDENCIA TÉCNICA Y RESULTADOS

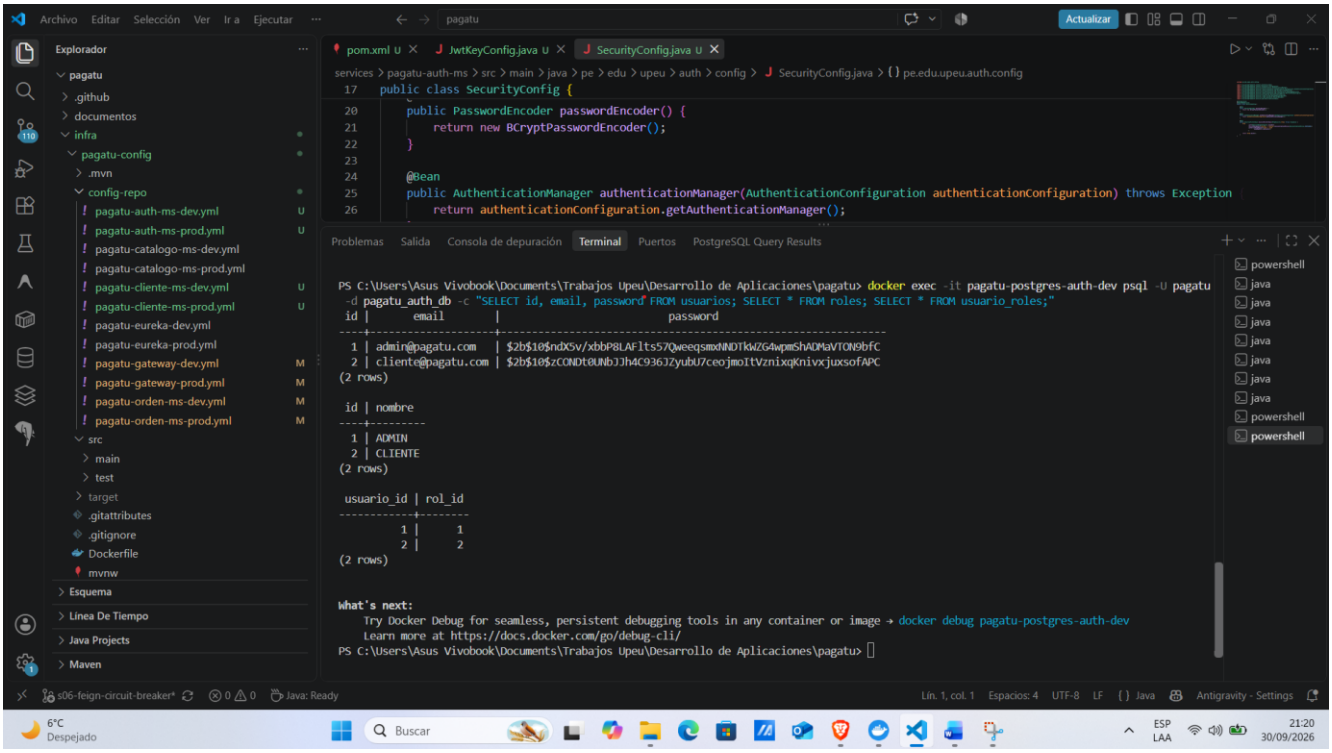
Dimensión 1: pagatu-auth-ms Construido y Registrado (Emisión de JWT y JWKS)

Levanté el microservicio pagatu-auth-ms en el puerto local 8085, vinculado a su base de datos PostgreSQL en Docker (contenedor pagatu-postgres-auth-dev en el puerto mapeado 15431, BD pagatu_auth_db). Durante el arranque, el servicio jala su configuración desde pagatu-config (http://localhost:18888/pagatu-auth-ms/dev) y se registra automáticamente en Eureka (http://localhost:18761).

Para la seguridad, habilité el endpoint /.well-known/jwks.json que expone únicamente la clave pública en formato JWKS. En el flujo de login (POST /api/v1/auth/login), el microservicio verifica las credenciales contrastándolas con los hashes BCrypt guardados en la base de datos; al autenticar con éxito, genera el JWT firmado con RS256 con los claims base (sub, iss, exp) y los personalizados (realm_access.roles e idCliente). También dejé funcionando el registro (POST /api/v1/auth/registro), el cual guarda los usuarios nuevos asignándoles el rol CLIENTE y encriptando sus contraseñas con BCrypt.


Informe S07 Seguridad
Portal Alumno - Entor
Pagatu - S7 - Seguridad
WhatsApp
Gym Class Heroes
como le pido a gimini
Eureka
localhost:8085/.well-k
localhost:18761
spring Eureka
HOME
LAST 1000 SINCE STARTUP
System Status
Environment
test
Data center
default
Current time
2026-09-30T21:09:32 -0500
Uptime
00:07
Lease expiration enabled
true
Renews threshold
10
Renews (last min)
20
THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS.
DS Replicas
localhost
Instances currently registered with Eureka
Application
AMIs
Availability Zones
Status
PAGATU-AUTH-MS
n/a (1)
(1)
UP (1) - pagatu-auth-ms:8085
PAGATU-CATALOGO-MS
n/a (1)
(1)
UP (1) - pagatu-catalogo-ms:8080
PAGATU-CLIENTE-MS
n/a (1)
(1)
UP (1) - pagatu-cliente-ms:8084
6°C
Despejado
Buscar
ESP
LAA
21:12
30/09/2026


Informe S07 Seguridad
Portal Alumno - Entor
Pagatu - S7 - Seguridad
WhatsApp
Gym Class Heroes
como le pido a gimini
Eureka
localhost:8085/.well-k
localhost:8085/.well-known/jwks.json
Dar formato al texto
{
"keys": [
{
"kty": "RSA",
"e": "AQAB",
"kid": "0f1d94dd-9c82-45a9-88b4-0dec163f55c1",
"n": "hG1__UB8BFx8AO7svbvlwk1td4FaG9DXul0o5b7906UyHryEiAuewE6-JPThdVknIm_ELfYPWLeAev2ITCqd7iOPfy3OOqch5WN0KwqnBa6aSW8TwVTiS6O0QVAxOgWr9CTZHCzQQJ0ktLUgrADCDeYucW2-xmlQjXSRD9cyPI0bvIlmaaZiKBH1ms1uNajdkXEL8D0VAmGDW2HpwR_FHSiv_gDMfeqtg3Mn4vNSKUIHr93WKttuMc56ZJUWHbgZVTMupQcFrjTHJMnv-A130PKusFPjsJXfqUENxnHQs_GSH5hvxxEFOP8-fj5wwo2i7FCMoLgSvmZcLheHlNXXxw"
}
]
}
6°C
Despejado
Buscar
ESP
LAA
21:13
30/09/2026

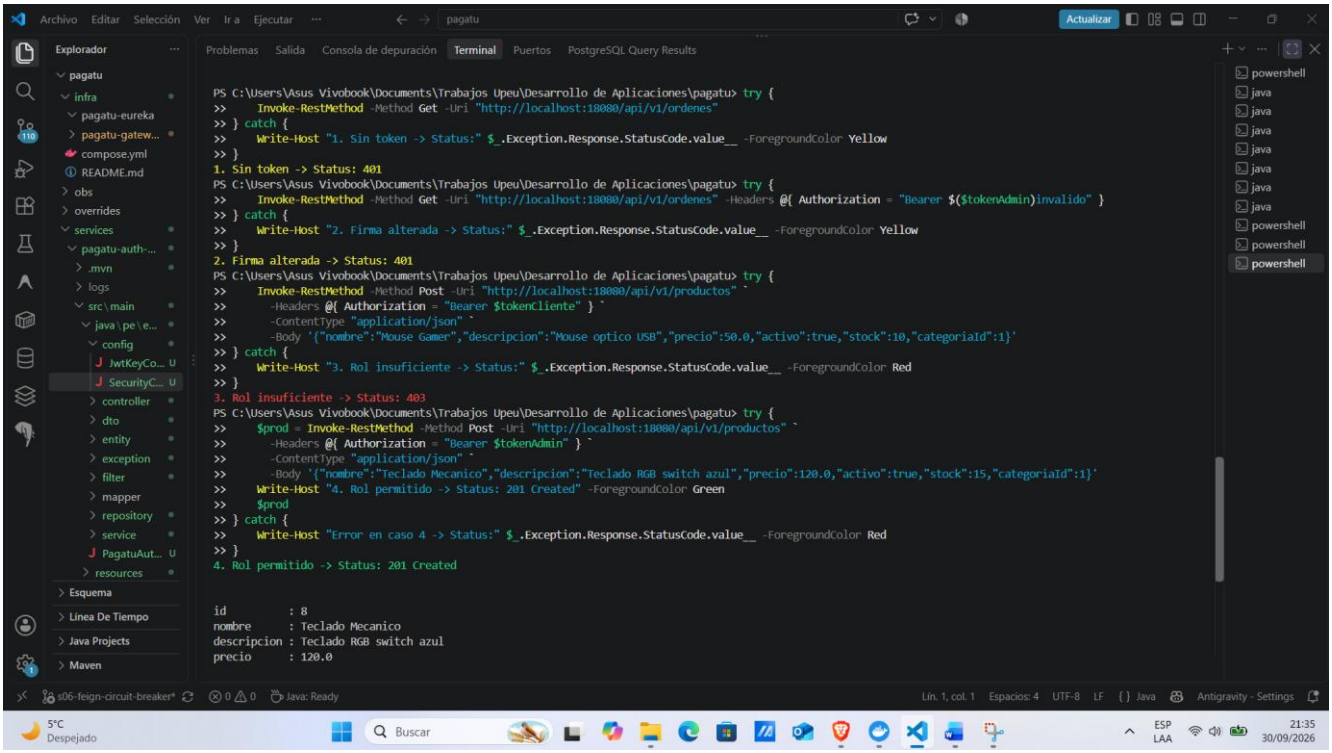


Dimensión 2: pagatu-gateway como Resource Server y Control de Acceso RBAC

Agregué la dependencia spring-boot-starter-security-oauth2-resource-server en pagatu-gateway (corriendo en el puerto 18080) y apunté la propiedad jwk-set-uri hacia las claves públicas de pagatu-auth-ms. Creé un JwtAuthenticationConverter personalizado para que Spring Security reconozca los roles anidados dentro del claim realm_access.roles y les anteponga el prefijo ROLE_.

Hice las pruebas de autorización para comprobar el funcionamiento del control de acceso:

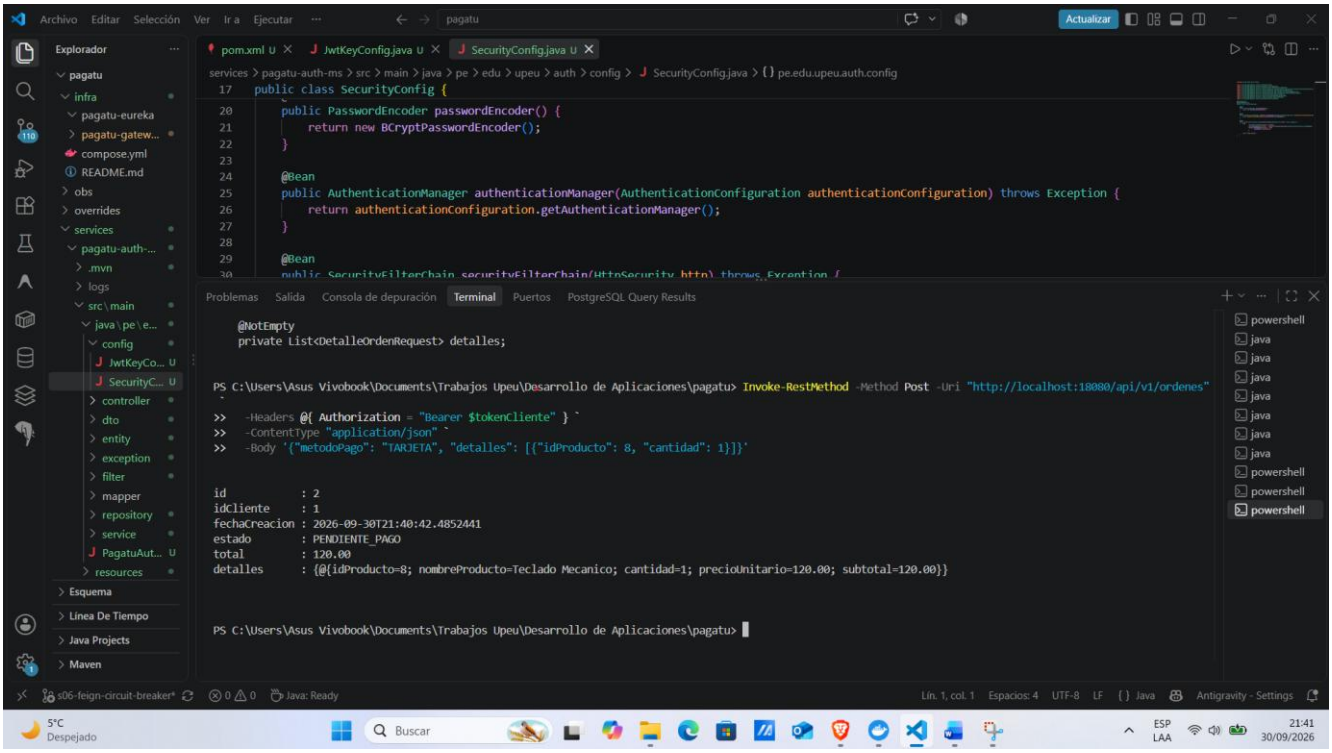
- **Sin Token:** Al consultar GET /api/v1/ordenes sin mandar el encabezado Authorization, el Gateway rechaza la petición con 401 Unauthorized.
- **Token con Firma Alterada:** Si altero un solo caracter de la firma del token, el Gateway detecta la inconsistencia con la clave pública y devuelve 401 Unauthorized.
- **Rol Insuficiente:** Al intentar crear un producto con un usuario de rol CLIENTE (POST /api/v1/productos), el sistema responde 403 Forbidden ya que la ruta requiere rol ADMIN.
- **Rol Permitido:** Haciendo la misma petición con un token con rol ADMIN, la operación se completa correctamente con código 201 Created.



Dimensión 3: pagatu-orden-ms como Resource Server y Extracción de idCliente

Había un problema de seguridad importante: el cliente podía mandar cualquier idCliente en el JSON y suplantar a otro usuario. Para solucionarlo, eliminé el campo idCliente del DTO OrdenRequest y configuré pagatu-orden-ms (puerto 8082) como su propio Resource Server para que verifique de forma independiente el token contra el JWKS.

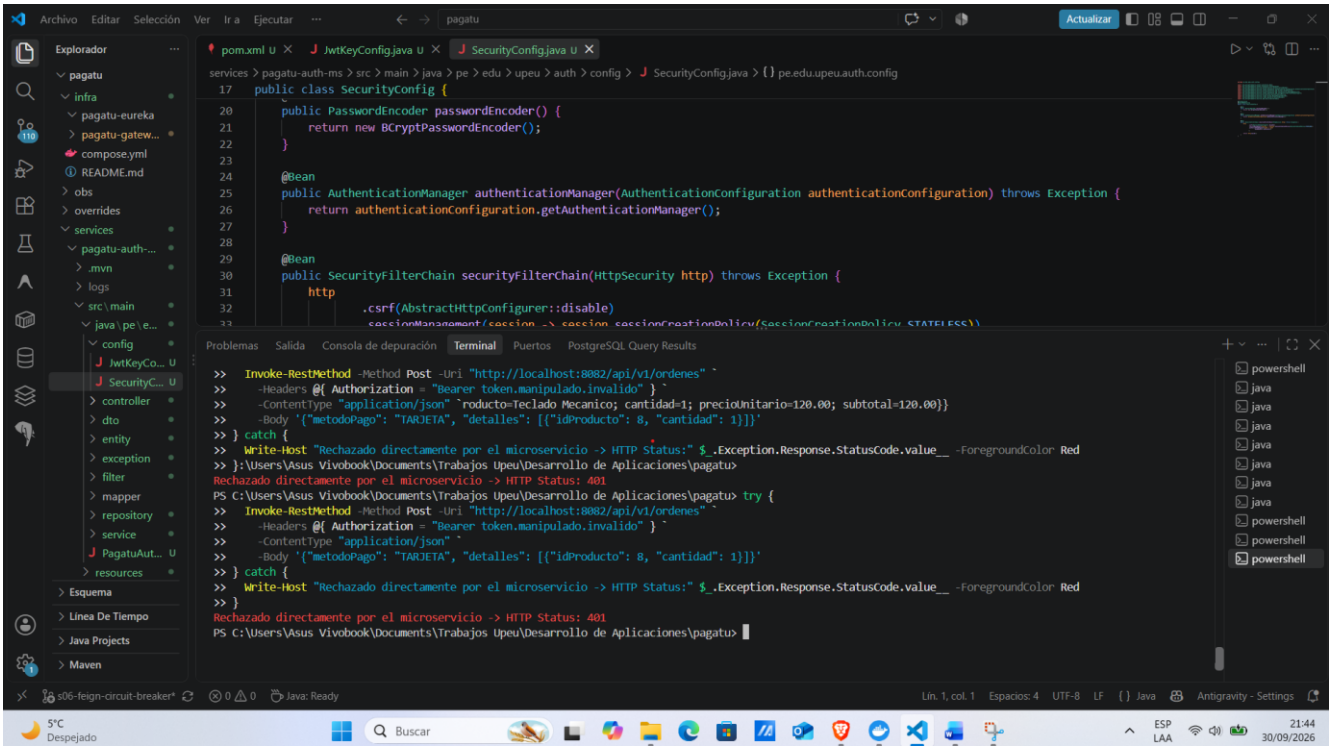
Dentro de OrdenController inyecté @AuthenticationPrincipal Jwt jwt para sacar el idCliente directo del token firmado. Ese valor es el que se pasa a la lógica de negocio para crear la orden. Si entra un token sin ese claim (como un admin), el código lo detecta y corta la petición arrojando un 400 Bad Request.



Dimensión 4: Defensa en Profundidad (Llamado Directo sin Gateway)

Para comprobar que no dependemos ciegamente del Gateway, probé llamar de forma directa al microservicio en su puerto local (POST http://localhost:8082/api/v1/ordenes). Al enviar la petición sin token o con un token dañado, el propio pagatu-orden-ms interceptó la llamada y respondió 401 Unauthorized (WWW-Authenticate: Bearer error="invalid_token").

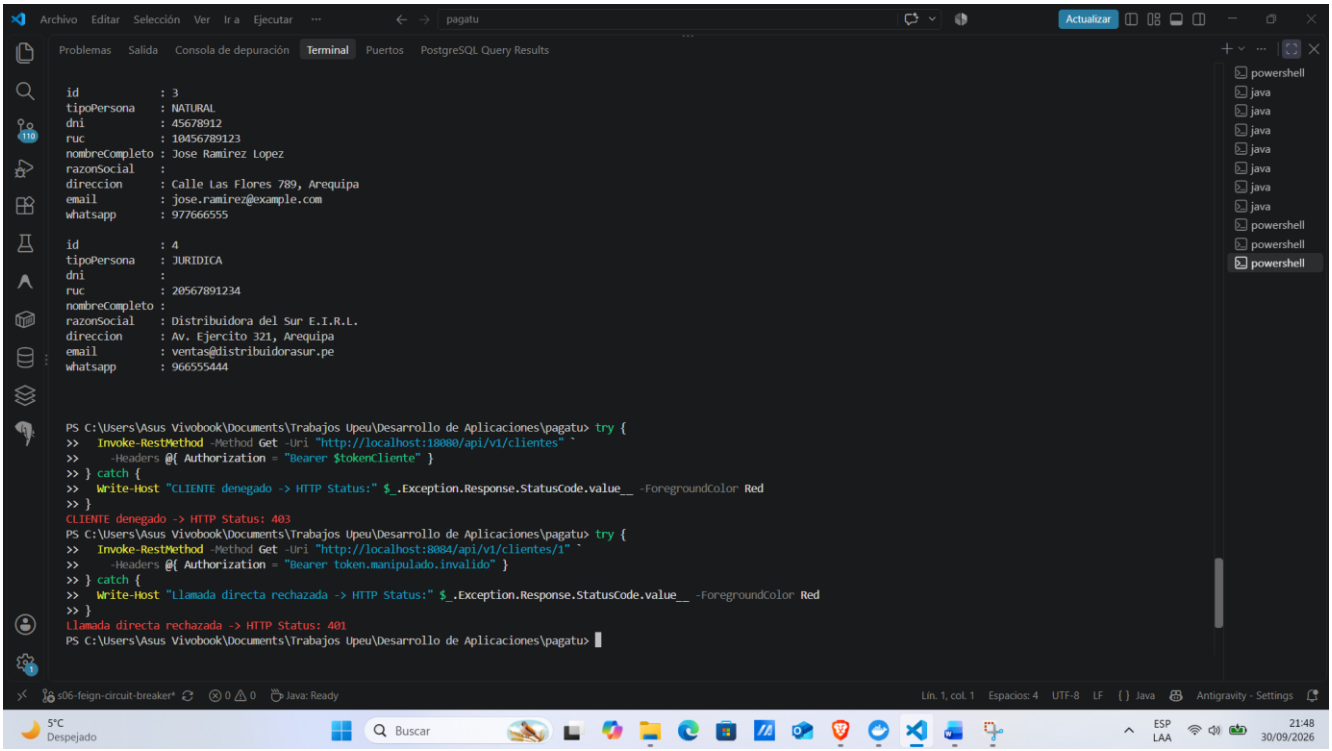
Con esto se demuestra el principio de defensa en profundidad: aunque alguien logre saltarse el Gateway y entrar a la red interna, cada servicio sigue protegido de manera autónoma validando la firma del token con la clave pública.



Dimensión 5: Microservicio pagatu-cliente-ms Protegido (Actividad Autónoma)

Apliqué el mismo esquema de seguridad en pagatu-cliente-ms para que no quedara desprotegido:

- Incluí la dependencia de oauth2-resource-server en el archivo pom.xml.
- Configuré el jwk-set-uri en pagatu-cliente-ms-dev.yml apuntando al servicio de autenticación.
- Añadí la clase SecurityConfig activando la validación JWT con oauth2ResourceServer.
- Ajusté las rutas en pagatu-gateway hacia lb://pagatu-cliente-ms, restringiendo GET /api/v1/clientes solo para administradores y dejando la consulta de perfiles abierta para CLIENTE y ADMIN.



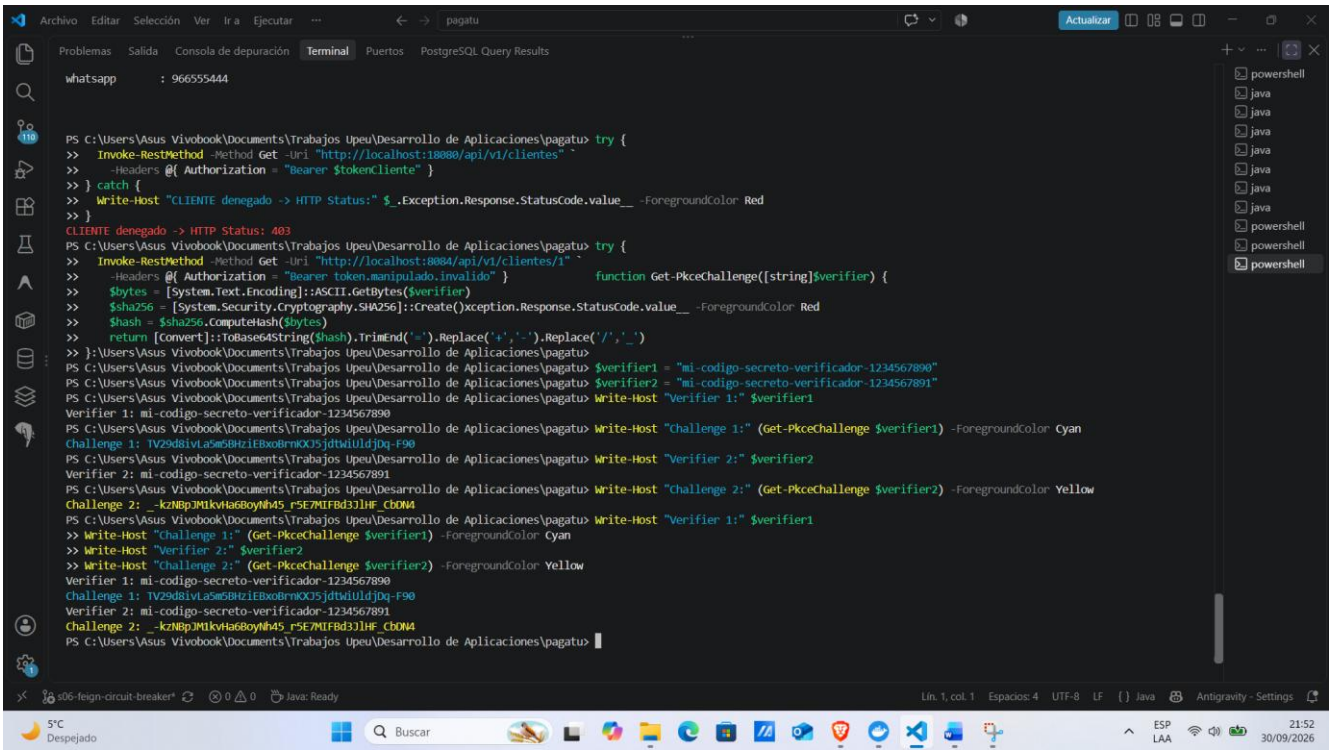
Matriz de Roles y Accesos Documentada

Ruta	Método	Rol Requerido	Sin Token	Token ADMIN	Token CLIENTE
/api/v1/auth/login	POST	Público	**200 OK**	**200 OK**	**200 OK**
/api/v1/auth/registro	POST	Público	**201 Created**	**201 Created**	**201 Created**
/api/v1/productos/**	GET	Público	**200 OK**	**200 OK**	**200 OK**
/api/v1/productos/**	POST / PUT / DELETE	ADMIN	**401 Unauthorized**	**201 / 200**	**403 Forbidden**
/api/v1/ordenes/**	POST	CLIENTE (o ADMIN con idCliente)	**401 Unauthorized**	**400 Bad Request**	**201 Created**
/api/v1/ordenes/**	GET	CLIENTE o ADMIN	**401 Unauthorized**	**200 OK**	**200 OK**
/api/v1/clientes	GET	ADMIN	**401 Unauthorized**	**200 OK**	**403 Forbidden**
/api/v1/clientes/**	GET	CLIENTE o ADMIN	**401 Unauthorized**	**200 OK**	**200 OK**

Dimensión 6: Verificación de Estado Stateless y Ejercicio PKCE

Comportamiento Stateless y Revocación: Borré manualmente la relación de roles del usuario en la base de datos (DELETE FROM usuario_roles). A pesar de esto, el token generado previamente siguió permitiendo crear órdenes (201 Created) hasta que venció su tiempo de vida, comprobando que los microservicios no guardan estado en sesión. Recién al hacer un login nuevo, el JWT se emitió sin roles y el Gateway lo bloqueó con 403 Forbidden.

Cálculo de PKCE: Probé el script de PowerShell para obtener el code_challenge mediante SHA-256 en Base64 URL-safe a partir del code_verifier. Al modificar un solo carácter de la cadena original, el resultado cambió completamente, comprobando el efecto avalancha que protege el intercambio del código en clientes públicos.



3. ERROR O HALLAZGO TÉCNICO

¿Qué pasó en la práctica?: Cuando quise probar los endpoints protegidos pasando por el Gateway, todas las peticiones me rebotaban con 401 Unauthorized, a pesar de que el token lo acababa de generar desde pagatu-auth-ms y parecía válido.

¿Por qué fallaba?: Activé los logs en modo DEBUG para Spring Security (logging.level.org.springframework.security: DEBUG) y vi que la firma no coincidía con el kid de la clave pública disponible. Me di cuenta de que había reiniciado pagatu-auth-ms para aplicar un cambio menor en el código justo después de haber sacado el token. Como en este laboratorio las llaves RSA se generan en memoria al levantar la aplicación (KeyPairGenerator), cada reinicio crea un par de llaves nuevo con otro kid, dejando automáticamente inservibles los tokens generados antes del reinicio.

¿Cómo lo solucioné?: Generé un nuevo token mediante POST /api/v1/auth/login sin reiniciar el servicio de autenticación y las peticiones pasaron con éxito. Esto me dejó claro por qué en proyectos reales se usa Keycloak u otro Identity Provider formal, donde las llaves se guardan en base de datos de forma persistente y no se pierden al reiniciar.

4. REFLEXIÓN TÉCNICA

(Respuesta en 5 a 8 líneas al requerimiento de la sección 4.3.1 de la guía)

Si más adelante migramos de pagatu-auth-ms a Keycloak, no tendríamos que tocar el código ni la lógica de permisos en el Gateway ni en los microservicios (orden y cliente), ya que todos están construidos bajo el estándar de OAuth2 Resource Server y el claim realm_access.roles sigue el estándar OIDC. La única modificación sería cambiar en la configuración YAML del Config Server la propiedad jwk-set-uri por issuer-uri para aprovechar el autodescubrimiento. Aquí se nota claramente la ventaja de usar criptografía asimétrica (RS256): el servidor de autenticación es el único que tiene la clave privada para firmar, mientras que los microservicios solo leen la pública por red para validar. Si usáramos una clave simétrica compartida (HS256), cualquier microservicio comprometido expondría el secreto y un atacante podría falsificar tokens de administrador para todo el ecosistema.

5. ANEXO: FEEDBACK DE LA SESIÓN

¿Cuál es el aprendizaje más importante que te llevas de la clase de hoy?

Entender cómo separar realmente la autenticación de la autorización en una arquitectura distribuida, usando claves públicas para validar tokens en cada servicio y evitando mandar datos sensibles o manipulables como idCliente en el body.

¿Qué punto de la clase te resultó más confuso o te dejó con dudas?

La configuración del JwtAuthenticationConverter para mapear la estructura anidada de realm_access.roles y lograr que Spring Security los reconozca con el prefijo ROLE_.

¿Tienes alguna pregunta que te gustaría que sea respondida la siguiente clase?

¿Cómo se maneja en la práctica una lista de revocación (blacklist) con Redis cuando necesitamos dar de baja un token JWT antes de que termine su tiempo de expiración?

Sobre tu nivel de comprensión de la clase de hoy:

[X] ¡Entendido! - Lo domino y podría explicarlo.

[ ] Más o menos. - Entendí la idea general, pero tengo dudas.

[ ] Necesito ayuda. - Me siento perdido/a con este tema.

¿Cómo puedo ayudarte a comprender mejor el tema?

Haciendo algún ejemplo de integración con Keycloak corriendo en Docker y conectándolo a un cliente en Angular con el flujo Authorization Code + PKCE.

Pensando en tu participación y esfuerzo en la clase de hoy, ¿cómo te autoevaluarías?

[X] Muy Comprometido/a: Me esforcé al máximo.

[ ] Comprometido/a: Sé que podría haberme esforzado un poco más.

[ ] Poco Comprometido/a: Hoy no di mi mejor esfuerzo.

Mi satisfacción con la clase fue (del 1 al 10):

Mi satisfacción fue de 9.